

Demonstration Video Walkthrough: System Features & Behaviors

This document provides a step-by-step technical breakdown of exactly what is happening in the Protoplasm PoC simulation, translating the visual behaviors into the underlying codebase mechanics. This is perfect for accompanying your video submission or adapting into your proposal.

Phase 1: Initialization & Staggered Takeoff

What is shown: The Pygame Tactical Dashboard initializes, showing the 1000x1000m Gazebo arena. The 10 drones appear at the GCS spawn point (World X: -575) and take off.

How it works in the code:

- `swarm.launch.py` staggers the boot sequence. It launches the MicroXRCE-DDS agent, then the PX4 SITL physics instances one by one (to prevent CPU overload), and finally the `drone_agent.py` ROS 2 nodes.
- The drones initialize their `PX4Bridge`, shift into Offboard mode, and autonomously ascend to the 30m AGL hover altitude defined in `swarm_params.yaml`.

Phase 2: Autonomous Exploration (The **SPREAD** Regime)

What is shown: The drones fan out rapidly across the arena, actively avoiding each other and spreading to maximize map coverage.

How it works in the code:

- The drone agents operate in a 20Hz Sense-Decide-Move loop.
- In the **SPREAD** regime, flight velocity is a calculated blend of a random-walk vector and a **Pheromone Gradient Pull**.
- As drones fly, they deposit "virtual pheromones" (via the `/swarm/deposits` topic) which slowly decay over time. Drones calculate the gradient of this shared field and are mathematically repulsed by it. This ensures they organically seek out unexplored areas of the map without needing a central path-planner.
- **Collision Avoidance:** During flight, if any drone detects a peer within 25m via telemetry, a hard inverse-distance repulsive force ($k = 5.0$ m/s) is injected into the velocity vector, guaranteeing the 20m minimum separation requirement.

Phase 3: Sensor Fusion & Target Convergence (The **CONVERGE** Regime)

What is shown: A drone flies near a hidden Point of Interest (POI). Instead of flying past, it slows down, adjusts its trajectory, and homes in on the target.

How it works in the code:

- `sensor_fusion.py` continuously samples the environment in a 3x3 grid below the drone. It applies a **Vantage Point Modifier**—meaning sensor strength depends realistically on the drone's angle and distance to the target.
- The system uses a strict **Cross-Modal Gate**: it requires *both* Thermal and RGB Camera signatures to simultaneously exceed a 0.5 threshold to filter out noise (like the Hot Debris and Wind Noise zones placed in the simulation).
- As confidence rises, the drone enters the **CONVERGE** regime. Its "viscosity" increases, causing its flight speed to drop from 5.0 m/s to 3.5 m/s, locking its gradient pull directly toward the signal peak.

Phase 4: Stigmergic Verification (The Sentinel-Verifier Handshake)

What is shown: A drone finds a survivor and stops moving. A short time later, a second drone breaks off from its search pattern, flies to the first drone, hovers nearby, and then the POI is officially confirmed.

How it works in the code:

- **The Sentinel:** When a drone's sensor confidence hits >0.65 , it shifts into the **SOLIDIFY** regime (slowing to a 1.0 m/s hover). It claims the role of **SENTINEL** and broadcasts a `VERIFY_REQUEST` over the 100m-limited `/swarm/radio` topic.
- **The Verifier:** A nearby drone in the **SPREAD** regime hears this request. It accepts the mission, becoming a **VERIFIER**. It overrides its exploration logic, flies directly to the Sentinel's coordinates, and takes its own independent sensor readings.
- If the Verifier also confirms the Thermal + RGB signatures, it broadcasts a `VERIFY_CONFIRM` message, preventing false positives from a single faulty drone.

Phase 5: Multi-Hop Daisy-Chain Relay (The **RELAY** Regime)

What is shown: On the dashboard, a verified POI turns Gold (Swarm Known). Drones reposition themselves, and suddenly a link connects them to the GCS, turning the POI Neon Green (GCS Confirmed) despite the POI being 800+ meters away from the GCS.

How it works in the code:

- The competition strictly enforces a 100m radio range limit. The GCS (at $x=-575$) cannot hear the drones deep in the arena.

- The swarm handles this via an emergent daisy-chain relay. When a `VERIFY_CONFIRM` happens, the swarm initiates a **Global 2-Phase Election**. The drone geographically closest to the GCS claims the `RELAY` role.
- The relay drone receives the `SURVIVOR_FOUND` message, checks its `relayed_pois` set (to prevent broadcast storms), **rewrites the sender coordinates to its own current position**, increments the hop count, and re-broadcasts it.
- By chaining these re-broadcasts, the message physically hops from drone to drone until it falls within 100m of the GCS, satisfying the BVLOS communication requirement dynamically.

Phase 6: Graceful Degradation & Return-to-Base (RTB)

What is shown: A drone abruptly stops exploring, turns around, and flies in a straight line back to the GCS to land. The rest of the swarm ignores its absence and seamlessly re-covers the gap it left behind.

How it works in the code:

- The drone agent actively monitors its battery capacity (45 minutes).
- When the battery drops to 20%, a hard safety override shifts the drone into the `RESCUED` (Return-to-Base) regime. This preempts all other behaviors—even if the drone was in the middle of verifying a survivor.
- The drone calculates a direct vector to its spawn point and lands.
- Because the swarm uses the decentralized pheromone field instead of assigned flight paths, the remaining active drones naturally sense the lack of pheromones in the returning drone's sector and drift over to cover the gap. No central reconfiguration or re-tasking is required.